

Тестовое задание на позицию Low-code программиста

Verba Group

ФИО: Язев Ярослав Викторович Телефон: +7 918 180 04 19

Задание 1. Логика работы голосового робота

Подход

Машина состояний + обработка реплик по интентам. Так читается лучше, чем вложенные if/else, и корректно работает с репликами, где несколько намерений сразу например, «Да, приду. А какой адрес?».

Допущения

1. CONFIRM / DENY — универсальные «да»/«нет» на текущий вопрос робота. Смысл зависит от контекста: в основном цикле — «да, приду», в handleCancel — «да, отменяйте», в handleReschedule — «да, соединяйте».
2. В реальной системе контекст уточнения хранится в session.pendingQuestion, а NLU учитывает его при распознавании. Здесь под-диалоги показаны явно, чтобы не раздувать пример.
3. Перенос не делаем сами — передаём оператору. Подбор слота не задача робота.
4. Мед. данные не раскрываем, пока не убедились, что попали на нужного человека.
5. Молчание / сброс звонка: таймаут ASR → тот же handleFallback.

Псевдокод

```
const Intent = {
  CONFIRM: 'CONFIRM',
  DENY: 'DENY',
  CANCEL: 'CANCEL',
  RESCHEDULE: 'RESCHEDULE',
  ASK_TIME: 'ASK_TIME',
  ASK_ADDRESS: 'ASK_ADDRESS',
  ASK_DOCTOR: 'ASK_DOCTOR',
  WRONG_NUMBER: 'WRONG_NUMBER',
};

const QUESTION_HANDLERS = {
  [Intent.ASK_TIME]: (a) => say(`Ваша запись на ${a.date} в ${a.time}.`),
  [Intent.ASK_ADDRESS]: (a) => say(`Адрес: ${a.clinic}.`),
  [Intent.ASK_DOCTOR]: (a) => say(`Вас принимает ${a.doctor}, ${a.specialty}.`),
};

const DECISION_HANDLERS = {
  [Intent.CANCEL]: handleCancel,
```

```
[Intent.RESCHEDULE]: handleReschedule,
[Intent.CONFIRM]:    handleConfirm,
};

// Если в реплике несколько решений берём первое по списку.
// Отмена приоритетнее переноса, перенос приоритетнее подтверждения.
const DECISION_PRIORITY = [Intent.CANCEL, Intent.RESCHEDULE, Intent.CONFIRM];

async function startCall(appointment, session) {
  await identifyPatient(appointment, session);
  if (session.finished) return;

  say(`Напоминаю: ${appointment.date} в ${appointment.time} вы записаны к
  ${appointment.doctor}, ${appointment.specialty}, ${appointment.clinic}.`);
  say('Подтверждаете запись?');

  while (!session.finished) {
    const { intents, confidence } = await listenAndDetect();

    if (isUnrecognized(intents, confidence)) {
      handleFallback(session);
      continue;
    }

    // Проверяем до раскрытия мед. данных: если номер ошибочный,
    // нельзя вслух называть ФИО врача и время приёма
    if (intents.includes(Intent.WRONG_NUMBER)) {
      handleWrongNumber(session);
      continue;
    }

    session.fallbackCount = 0;

    // Сначала вопросы пациент ждёт ответа сейчас.
    // Потом решения они меняют ход диалога.
    const askedQuestion = answerQuestions(intents, appointment);
    const decision = DECISION_PRIORITY.find((i) => intents.includes(i));

    if (decision) {
      await DECISION_HANDLERS[decision](session, appointment);
      continue;
    }

    if (intents.includes(Intent.DENY)) {
      say('Понял. Хотите отменить запись или перенести на другое время?');
      continue;
    }

    // Был только вопрос, после ответа возвращаемся к сценарию
    if (askedQuestion) {
      say('Подтверждаете запись?');
      continue;
    }
  }
}
```

```
    handleFallback(session);
  }
}

function answerQuestions(intents, appointment) {
  const questions = intents.filter((i) => QUESTION_HANDLERS[i]);
  questions.forEach((i) => QUESTION_HANDLERS[i](appointment));
  return questions.length > 0;
}

function isUnrecognized(intents, confidence) {
  return intents.length === 0 || confidence < 0.5;
}

// Хендлеры решений
function handleConfirm(session, appointment) {
  say(`Отлично, запись подтверждена. Ждём вас ${appointment.date} в ${appointment.time}.`);
  finish(session);
}

async function handleCancel(session, appointment) {
  // Подтверждение защита от случайного «отмена»
  say(`Вы хотите отменить запись к ${appointment.doctor} на ${appointment.date} ${appointment.time}. Подтверждаете?`);

  const { intents, confidence } = await listenAndDetect();

  if (isUnrecognized(intents, confidence)) return handleFallback(session);
  session.fallbackCount = 0;

  answerQuestions(intents, appointment);

  if (intents.includes(Intent.CONFIRM)) {
    cancelAppointment(appointment.appointmentId);
    say('Запись отменена. ');
    return finish(session);
  }

  if (intents.includes(Intent.DENY)) {
    say('Хорошо, оставляю запись в силе. Подтверждаете запись?');
    return;
  }

  // Счётчик растёт только здесь, чтобы вопросы не считались непониманием
  session.fallbackCount += 1;
  if (session.fallbackCount >= 2) return handleFallback(session);
  say('Уточните, пожалуйста: отменяем запись или оставляем?');
}

async function handleReschedule(session, appointment) {
  say('Понял, хотите перенести запись. Соединяю с администратором – он подберёт удобное время. Хорошо?');

  const { intents, confidence } = await listenAndDetect();
```

```

    if (isUnrecognized(intents, confidence)) return handleFallback(session);
    session.fallbackCount = 0;

    answerQuestions(intents, appointment);
    if (intents.includes(Intent.CONFIRM)) {
        say('Перезову на администратора.');
```

transferToOperator();

```
        return finish(session);
    }

    if (intents.includes(Intent.DENY)) {
        say('Хорошо, оставляю запись в силе. Подтверждаете запись?');
        return;
    }
    session.fallbackCount += 1;
    if (session.fallbackCount >= 2) return handleFallback(session);
    say('Уточните, пожалуйста: соединяем с администратором или оставляем запись?');
}
// Fallback и краевые случаи
function handleWrongNumber(session) {
    say('Извините за беспокойство, мы ошиблись номером. Всего доброго.');
```

finish(session);

```

}
function handleFallback(session) {
    session.fallbackCount += 1;
    // После двух неудач не гадаем передаём администратору,
    // иначе можно случайно отменить или подтвердить запись за пациента
    if (session.fallbackCount >= 2) {
        say('Извините, не могу разобрать. Передам администратору, он свяжется с вами.
До свидания.');
```

return finish(session);

```
    }
    say('Извините, я не совсем понял. Вы подтверждаете запись, хотите отменить или
перенести?');
```

}

```

function finish(session) {
    session.finished = true;
}
// Заглушки в реальности это телефония и NLU
function say(text) { /* TTS */ }
async function listenAndDetect()
async function identifyPatient(appointment, session)
function cancelAppointment(id)
function transferToOperator()

```

Задание 1. JavaScript

Задача 1. Формирование текста для работа

```

```js
function formatAppointmentText(appointment) {
 const { patientName, doctor, specialty, date, time } = appointment;

```

```
// Разбиваю дату вручную: new Date('2026-10-12') парсится как UTC,
// и в некоторых часовых поясах дата может съехать на день назад
const [year, month, day] = date.split('-');
const formattedDate = `${day}.${month}.${year}`;

return `${patientName}, вы записаны к врачу ${doctor}, ${specialty}, на
${formattedDate} в ${time}.`;
}

// Проверка
formatAppointmentText({
 patientName: 'Анна',
 doctor: 'Иванов Сергей Петрович',
 specialty: 'терапевт',
 date: '2026-10-12',
 time: '15:30',
});
// → "Анна, вы записаны к врачу Иванов Сергей Петрович, терапевт, на 12.10.2026 в
15:30."
```js
```

Задача 2.

```
```js
function formatAppointments(appointments) {
 // map, а не forEach + push: получаем новый массив,
 // исходный response не мутируется
 return appointments.map(({ doctor, time }) => `${doctor} - ${time}`);
}
```

Проверка

```
const response = {
 success: true,
 appointments: [
 { id: 'A-101', doctor: 'Иванов', time: '10:00' },
 { id: 'A-102', doctor: 'Петров', time: '14:30' },
],
};
```

```
formatAppointments(response.appointments);
// ["Иванов - 10:00", "Петров - 14:30"]
```js
```

Задание 3. Промпт для LLM

Ты – модуль извлечения структурированных данных для голосового робота аптеки. На вход подаётся свободная реплика клиента. Твоя задача – извлечь параметры товара и вернуть строго JSON по заданной схеме. Без пояснений, markdown, code fence и любого текста вокруг JSON.

СХЕМА ОТВЕТА (все ключи обязательны, значения могут быть null):

```
{
  "product": string | null,
  "dosage": string | null,
```

```
"form": string | null,  
"package": string | null  
}
```

ПРАВИЛА:

1. **product** – название препарата или действующего вещества, включая уточнения («детский», «форте», «актив»), но без дозировки, формы и фасовки.
Если клиент называет только симптом или задачу («что-нибудь от аллергии», «от головной боли») и конкретный товар не назван – **product** = **null**.
2. **dosage** – дозировка или концентрация действующего вещества.
Примеры: «500 мг», «0,5 г», «10 мг/мл», «100 мг/5 мл».
Нормализуй «миллиграмм» → «мг», «грамм» → «г», если это однозначно.
Не путай **dosage** с объёмом упаковки. Если дозировка не названа – **null**.
3. **form** – лекарственная форма: «таблетки», «капсулы», «сироп», «капли», «спрей», «мазь», «свечи» и т.п.
Если форма явно следует из фасовки или единиц («20 таблеток» → «таблетки»), её можно указать. Если форма не названа и не следует из текста – **null**.
4. **package** – фасовка или размер упаковки: «20 таблеток», «упаковка на 20 таблеток», «100 мл», «большой флакон», «№20».
Для «упаковка на 20 таблеток» верни **package** = «20 таблеток».
Не путай **package** с **dosage**. Если фасовка не названа – **null**.
5. НЕ придумывай данные, которых нет в реплике.
Если значение не названо и не следует однозначно из текста – **null**.
6. Если реплика пустая, содержит только шум («алло», «ау», неразборчиво) или не относится к поиску товара – верни все поля **null**.
7. Не добавляй поля, которых нет в схеме.
8. Верни ТОЛЬКО JSON. Без markdown, без пояснений, без текста до и после.

ПРИМЕРЫ:

Вход: «Мне нужен Нурофен детский, сироп, большой флакон»

Выход: {"product":"Нурофен детский","dosage":null,"form":"сироп","package":"большой флакон"}

Вход: «Есть у вас парацетамол 500 миллиграмм, упаковка на 20 таблеток?»

Выход: {"product":"Парацетамол","dosage":"500 мг","form":"таблетки","package":"20 таблеток"}

Вход: «Нужен парацетамол 500 мг»

Выход: {"product":"Парацетамол","dosage":"500 мг","form":null,"package":null}

Вход: «Парацетамол детский 100 мг/5 мл суспензия»

Выход: {"product":"Парацетамол детский","dosage":"100 мг/5 мл","form":"суспензия","package":null}

Вход: «Мне что-нибудь от аллергии»

Выход: {"product":null,"dosage":null,"form":null,"package":null}

Вход: «А есть капли в нос для ребёнка?»

Выход: {"product":null,"dosage":null,"form":"капли","package":null}

Где использовался Ai при решении тестового

Задача 2: решена самостоятельно, AI не использовался.

Задача 1: сценарий и краевые случаи разобрал сам. С подходом через интенды и таблицу хендлеров до этого не сталкивался: подход подсказал AI, я его разобрал, переписал под задачу и адаптировал – приоритет обработки, гейт на ошибочный номер, обработка комбинированной реплики («Да, приду. А какой адрес?») и счётчик непонимания. AI также помог с формулировками комментариев.

Задача 3: схему, поля и правила вывел из условия сам. AI использовал, чтобы найти двусмысленные формулировки в правилах промпта и переписать их; примеры собрал сам.